

```
#!/usr/bin/env python3
"""
XAU Sniper relay server (Python 3.8+,
standard library only).

Flow:
    MT5 EA --POST /push--> this server (keeps
the latest live snapshot)
    Claude (in chat, on your request) --GET
/latest?key=READ_KEY--> analyzes live data

Environment variables (both REQUIRED, use
long random strings):
    PUSH_KEY  secret used by the EA    -> EA
input InpApiToken
    READ_KEY  secret used by Claude    -> goes
inside the /latest link you paste in chat
    PORT      listen port (default 8080;
hosting platforms set it automatically)

Run:
    PUSH_KEY=abc... READ_KEY=xyz... python3
relay_server.py

Endpoints:
    POST /push                                header
Authorization: Bearer PUSH_KEY    body: JSON
from EA
    GET /latest?key=READ_KEY    compact
text snapshot, last 24/24/30/40 candles
```

```
(fast)
  GET /latest?key=READ_KEY&n=150 last n
candles per timeframe
  GET /latest?key=READ_KEY&format=json
raw JSON
  GET /health status +
snapshot age
```

Deploy on any host that gives you public HTTPS (Render, Railway, Fly.io, VPS + Caddy/nginx).

EA URL: <https://YOUR-HOST/push>

Chat URL: https://YOUR-HOST/latest?key=READ_KEY

"""

```
import hmac
import json
import os
import sys
import threading
import time
from datetime import datetime, timezone
from http.server import
BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import parse_qs, urlparse
```

```
PUSH_KEY = os.environ.get("PUSH_KEY", "")
READ_KEY = os.environ.get("READ_KEY", "")
PORT = int(os.environ.get("PORT", "8080"))
SNAPSHOT_FILE =
os.environ.get("SNAPSHOT_FILE",
"snapshot.json")
MAX_BODY = 3 * 1024 * 1024
```

```
LOCK = threading.Lock()
STATE = {"data": None, "received": 0.0}

def key_ok(given, expected):
    if not expected or not given:
        return False
    return
    hmac.compare_digest(given.encode("utf-8"),
expected.encode("utf-8"))

def save_snapshot():
    try:
        with open(SNAPSHOT_FILE, "w",
encoding="utf-8") as f:
            json.dump(STATE, f)
    except OSError:
        pass

def load_snapshot():
    try:
        with open(SNAPSHOT_FILE, "r",
encoding="utf-8") as f:
            d = json.load(f)
            if isinstance(d, dict) and "data"
in d:
                STATE.update(d)
    except (OSError, ValueError):
        pass

def fmt_ts(epoch):
```

```

        return
datetime.fromtimestamp(int(epoch),
tz=timezone.utc).strftime("%m-%d %H:%M")

DEFAULT_N = {"H1": 24, "M30": 24, "M15":
30, "M5": 40}

def render_text(d, received, n=None):
    age = int(time.time() - received)
    p = d.get("previous", {}) or {}
    out = []
    out.append(
        "SNAPSHOT symbol=%s server_time=%s
age_sec=%d"
        % (d.get("symbol"),
d.get("server_time"), age)
    )
    out.append(
        "bid=%s ask=%s pip_size=%s
sl_pips=%s min_rr=%s min_confidence=%s
max_limit_distance_pips=%s"
        % (
            d.get("bid"), d.get("ask"),
d.get("pip_size"), d.get("sl_pips"),
            d.get("min_rr"),
d.get("min_confidence"),
d.get("max_limit_distance_pips"),
        )
    )
    out.append(
        "prev_day_high=%s prev_day_low=%s
prev_week_high=%s prev_week_low=%s"

```

```

        % (p.get("day_high"),
p.get("day_low"), p.get("week_high"),
p.get("week_low"))
    )
    out.append(
        "candles oldest->newest | columns:
time(server),open,high,low,close,tick_volum
e | "
        "last_candle_forming=%s (last row
of each TF is the live, still-forming
candle)"
        % d.get("last_candle_forming")
    )
    candles = d.get("candles", {}) or {}
    for tf in ("H1", "M30", "M15", "M5"):
        rows = candles.get(tf, []) or []
        rows = rows[-(n or DEFAULT_N[tf]):]
        out.append("## %s (%d candles)" %
(tf, len(rows)))
        for r in rows:
            out.append("%s,%s,%s,%s,%s,%s"
% (fmt_ts(r[0]), r[1], r[2], r[3], r[4],
r[5]))
    return "\n".join(out)

class Handler(BaseHTTPRequestHandler):
    server_version = "XAURelay/1.0"

    def log_message(self, fmt, *args):
        # never log the query string (it
contains the read key)
        try:
            path = urlparse(self.path).path

```

```

        except Exception:
            path = "?"
            sys.stderr.write("%s %s %s\n" %
                (self.log_date_time_string(), self.command,
                path))

        def _send(self, code, body,
            ctype="application/json; charset=utf-8"):
            if isinstance(body, str):
                body = body.encode("utf-8")
            self.send_response(code)
            self.send_header("Content-Type",
            ctype)
            self.send_header("Content-Length",
            str(len(body)))
            self.send_header("Cache-Control",
            "no-store")
            self.end_headers()
            self.wfile.write(body)

        def _json(self, code, obj):
            self._send(code, json.dumps(obj))

        def do_POST(self):
            if urlparse(self.path).path !=
            "/push":
                return self._json(404,
                {"error": "not found"})
            auth =
            self.headers.get("Authorization", "")
            token = auth[7:] if
            auth.lower().startswith("bearer ") else ""
            if not key_ok(token, PUSH_KEY):
                return self._json(401,

```

```

{"error": "unauthorized"})
    try:
        n =
int(self.headers.get("Content-Length",
"0"))
    except ValueError:
        n = 0
        if n <= 0 or n > MAX_BODY:
            return self._json(413,
{"error": "bad size"})
        raw = self.rfile.read(n)
        try:
            data =
json.loads(raw.decode("utf-8"))
            if not isinstance(data, dict)
or "candles" not in data:
                raise ValueError("missing
candles")
        except (ValueError,
UnicodeDecodeError) as e:
            return self._json(400,
{"error": "invalid json: %s" % e})
        with LOCK:
            STATE["data"] = data
            STATE["received"] = time.time()
            save_snapshot()
        return self._json(200, {"ok":
True})

def do_GET(self):
    u = urlparse(self.path)
    q = parse_qs(u.query)
    if u.path == "/health":
        with LOCK:

```

```

        age = int(time.time() -
STATE["received"]) if STATE["data"] else
None
        return self._json(200, {"ok":
True, "snapshot_age_sec": age})
        if u.path == "/latest":
            given = (q.get("key", [""])[0])
or self.headers.get("X-Read-Key", "")
            if not key_ok(given, READ_KEY):
                return self._json(401,
{"error": "unauthorized"})
            with LOCK:
                data, received =
STATE["data"], STATE["received"]
                if not data:
                    return self._json(404,
{"error": "no snapshot yet - is the EA
running?"})
                if (q.get("format", [""])
[0]).lower() == "json":
                    return self._json(200,
{"age_sec": int(time.time() - received),
"data": data})
                try:
                    n = int(q.get("n", ["0"])
[0]) or None
                except ValueError:
                    n = None
                return self._send(200,
render_text(data, received, n),
"text/plain; charset=utf-8")
                return self._json(404, {"error":
"not found"})

```



```
def main():
    if not PUSH_KEY or not READ_KEY:
        sys.exit("Set PUSH_KEY and READ_KEY
environment variables.")
    if PUSH_KEY == READ_KEY:
        sys.exit("PUSH_KEY and READ_KEY
must be different.")
    load_snapshot()
    srv = ThreadingHTTPServer(("0.0.0.0",
PORT), Handler)
    print("XAU relay listening on port %d"
% PORT)
    srv.serve_forever()

if __name__ == "__main__":
    main()
```